

Week 8 Lab Assistant Prep - Raspberry Pi Performance & Metrics Lab

1. What Week 8 is about

Week 8 is about measuring Raspberry Pi performance.

Students are not building a big system this week. They are testing different parts of the Raspberry Pi and recording results:

- CPU speed
- Memory allocation
- Storage read/write speed
- Network bandwidth and latency
- Temperature and throttling under load

The main idea is: "Computers have different resources, and we can measure how well each resource performs."

This is a good week for connecting theory to real hardware. Students can actually see that a Raspberry Pi has limits: CPU gets busy, memory fills up, storage has read/write speed, network has latency, and temperature can rise under stress.

2. Relevant repo folders/files

The main folder is:

```
week08/
```

Files:

```
week08/README.md
week08/01_cpu.md
week08/02_memory.md
week08/03_io.md
week08/04_network.md
week08/05_thermal.md
```

Expected output folder:

```
results/
```

Expected student submission:

```
results/cpu.txt
results/memory.txt
results/io.txt
results/iperf.txt
results/ping.txt
results/thermal.txt
# or
results/temperature.csv
```

REPORT.md

Important: the repo asks students to upload a results/ folder and a short REPORT.md.

3. What students are expected to do

Students should run benchmark-style tests on the Raspberry Pi and save the outputs.

Task 1 - CPU test

They run a Python script that performs many calculations:

```
import time

def cpu_stress(n=5_000_000):
    total = 0
    for i in range(n):
        total += i*i
    return total

start = time.perf_counter()
cpu_stress()
end = time.perf_counter()
print('Execution time:', end - start, 'seconds')
```

Then they record:

```
vcgencmd measure_temp
htop
```

Save to:

```
results/cpu.txt
```

What they are measuring:

- How long the Pi takes to do repeated calculations
- CPU temperature
- CPU usage while the task runs

Task 2 - Memory allocation test

They run:

```
import time, numpy as np

start = time.perf_counter()
a = np.random.rand(10_000_000)
end = time.perf_counter()

print('Memory allocation time:', end - start)
```

Then check memory:

```
free -h
```

Save to:

```
results/memory.txt
```

What they are measuring:

- How long it takes to create a large array in memory
- How much RAM is used/free

Possible issue: numpy might not be installed.

Fix:

```
sudo apt update  
sudo apt install python3-numpy
```

or:

```
pip3 install numpy
```

Task 3 - Storage I/O test

Write test:

```
dd if=/dev/zero of=testfile.bin bs=100M count=1 oflag=direct
```

Read test:

```
dd if=testfile.bin of=/dev/null bs=100M count=1 iflag=direct
```

Save to:

```
results/io.txt
```

What they are measuring:

- Write speed to storage
- Read speed from storage

Important: dd often prints output to stderr, so simple > redirection may not capture the result properly.

Better command:

```
dd if=/dev/zero of=testfile.bin bs=100M count=1 oflag=direct 2>&1 | tee results/io.txt
```

Append the read test:

```
dd if=testfile.bin of=/dev/null bs=100M count=1 iflag=direct 2>&1 | tee -a results/io.txt
```

After the test, clean up:

```
rm testfile.bin
```

Task 4 - Network latency and bandwidth

On the server machine:

```
iperf3 -s
```

On the Raspberry Pi:

```
iperf3 -c <server-ip>  
ping <server-ip> -c 30
```

Save:

```
results/iperf.txt  
results/ping.txt
```

What they are measuring:

- Bandwidth: how much data can move per second
- Latency: how long packets take to travel
- Packet loss: whether packets are being dropped

Useful commands to find IP address:

On Linux/Raspberry Pi:

```
hostname -I
```

or:

```
ip addr
```

On macOS:

```
ipconfig getifaddr en0
```

For Wi-Fi, usually en0.

Possible issue: iperf3 may not be installed.

Fix:

```
sudo apt update  
sudo apt install iperf3
```

Task 5 - Temperature and throttling

Run stress test:

```
stress --cpu 4 --timeout 30
```

In another terminal:

```
watch -n 2 vcgencmd measure_temp
```

Save to:

```
results/thermal.txt
```

or:

```
results/temperature.csv
```

What they are measuring:

- How hot the Pi gets under CPU load
- Whether cooling is enough
- Whether performance may drop because of heat

Possible issue: stress may not be installed.

Fix:

```
sudo apt update  
sudo apt install stress
```

4. What you should personally understand before the lab

CPU

CPU means Central Processing Unit. It runs instructions and calculations.

Beginner explanation:

The CPU is like the brain of the computer. The faster it can process instructions, the faster programs can run.

In this lab, the Python loop does many multiplications:

```
total += i*i
```

Students should understand that a longer execution time usually means slower performance, but results can vary depending on:

- Raspberry Pi model
- CPU temperature
- Background processes
- Power supply quality
- Whether the Pi is throttling

Memory/RAM

RAM is temporary working memory.

Beginner explanation:

RAM is like the desk space of the computer. If the desk is big, the computer can keep more things open and work with bigger data.

The NumPy task creates a large array:

```
np.random.rand(10_000_000)
```

This creates 10 million floating-point numbers. It can use a lot of memory.

Students may ask: "Is this storage?"

Explain:

No. This array is created in RAM, not saved permanently to the SD card.

Storage I/O

Storage I/O means reading from and writing to the disk or SD card.

Beginner explanation:

Storage is like a notebook. Writing speed means how fast the computer can save data. Reading speed means how fast it can load data back.

On Raspberry Pi, storage speed often depends on the microSD card. A slow SD card can make the whole Pi feel slow.

Network performance

There are two important ideas:

Bandwidth:

How much data can be transferred per second.

Latency:

How long it takes for one message to travel from one device to another and back.

Simple example:

Bandwidth is like the width of a road. Latency is like how long it takes the first car to arrive.

iperf3 measures bandwidth.

ping measures latency and packet loss.

Temperature and throttling

When the Pi gets too hot, it may slow itself down to protect the hardware.

Beginner explanation:

If the Raspberry Pi gets too hot, it can reduce its speed automatically. This is called throttling.

Useful command:

```
vcgencmd measure_temp
```

For throttling status:

```
vcgencmd get_throttled
```

This is not in the repo, but it is useful for you as a lab assistant.

If it returns:

```
throttled=0x0
```

that means no throttling is currently detected.

5. Common student mistakes and how to fix them

Mistake 1 - They forget to create the `results/` folder

Error example:

```
bash: results/cpu.txt: No such file or directory
```

Fix:

```
mkdir -p results
```

Mistake 2 - They run Python code directly in the terminal

They may paste Python into Bash and get errors.

Fix: create a file first:

```
nano cpu_test.py
```

Paste the Python code, save, then run:

```
python3 cpu_test.py
```

Save output:

```
python3 cpu_test.py > results/cpu.txt
```

Mistake 3 - `numpy` is missing

Error:

```
ModuleNotFoundError: No module named 'numpy'
```

Fix:

```
sudo apt update  
sudo apt install python3-numpy
```

Mistake 4 - `stress`, `htop`, or `iperf3` missing

Fix:

```
sudo apt update  
sudo apt install stress htop iperf3
```

Mistake 5 - `vcgencmd` not found

Possible causes:

- They are not on a Raspberry Pi
- Raspberry Pi tools are missing
- They are using the wrong environment

Check:

```
which vcgencmd
```

On Raspberry Pi OS, it is usually available.

Mistake 6 - `iperf3` client cannot connect

Error:

```
iperf3: error - unable to connect to server
```

Possible fixes:

1. Make sure server is running:

```
iperf3 -s
```

2. Make sure both devices are on the same network.
3. Check server IP:

```
hostname -I
```

4. Test basic connection:

```
ping <server-ip>
```

5. Check firewall if using laptop as server.

Mistake 7 - Students confuse bandwidth and latency

Use this simple explanation:

Ping tells us how quickly a small message travels. Iperf tells us how much data can be transferred per second.

Example:

- Low ping = quick response
- High bandwidth = lots of data can move

They are related to network quality but they are not the same thing.

Mistake 8 - They redirect `dd` output but file is empty

This happens because dd prints to stderr.

Fix:


```
dd if=/dev/zero of=testfile.bin bs=100M count=1 oflag=direct 2>&1 | tee results/io.txt
```

Then append the read test:

```
dd if=testfile.bin of=/dev/null bs=100M count=1 iflag=direct 2>&1 | tee -a results/io.txt
```

Mistake 9 - They leave `testfile.bin` behind

This wastes storage.

Fix:

```
rm testfile.bin
```

Mistake 10 - They think benchmark results must be identical

Explain:

Results can be different between groups because of different SD cards, Wi-Fi strength, background processes, cooling, and Raspberry Pi model.

The goal is not perfect identical numbers. The goal is to measure, record, and explain.

6. Commands/tools/concepts you should be ready to explain

Folder and file setup

```
mkdir -p results
ls
pwd
nano cpu_test.py
python3 cpu_test.py
```

Save output to file

Overwrite:

```
command > file.txt
```

Append:

```
command >> file.txt
```

Capture errors/output too:

```
command 2>&1 | tee file.txt
```

Append with tee:

```
command 2>&1 | tee -a file.txt
```

CPU and system monitoring

```
htop  
vcgencmd measure_temp  
vcgencmd get_throttled
```

Memory

```
free -h
```

Explain columns simply:

- total = total RAM
- used = RAM currently used
- free = unused RAM
- available = RAM that can still be used by programs

Storage I/O

```
dd if=/dev/zero of=testfile.bin bs=100M count=1 oflag=direct  
dd if=testfile.bin of=/dev/null bs=100M count=1 iflag=direct  
rm testfile.bin
```

Simple explanation:

- `/dev/zero` produces empty zero data
- `of=testfile.bin` writes into a file
- `/dev/null` throws data away
- `bs=100M count=1` means write/read one block of 100 MB

Network

```
hostname -I  
ping <server-ip> -c 30  
iperf3 -s  
iperf3 -c <server-ip>
```

Thermal stress

```
stress --cpu 4 --timeout 30  
watch -n 2 vcgencmd measure_temp
```

7. Physical or software setup to check before the session

Hardware setup

Check:

- Raspberry Pis are powered properly
- Each group has a Pi
- Pi has network access
- Pi has keyboard/mouse/monitor or SSH access
- Pi has enough storage space

- Pi has cooling if available: heatsink/fan
- Power supply is stable

Useful command:

```
df -h
```

This checks disk space.

Software setup

Before the lab, check at least one Pi:

```
python3 --version
which vcgencmd
which htop
which iperf3
which stress
```

Install missing tools:

```
sudo apt update
sudo apt install htop iperf3 stress python3-numpy
```

Check Git is available:

```
git --version
```

Check repo structure:

```
ls week08
```

Check results folder:

```
mkdir -p results
```

Network setup

You need one machine to act as the `iperf3` server.

This can be:

- Your laptop
- Another Raspberry Pi
- A lab computer

On server:

```
iperf3 -s
```

On Pi:

```
iperf3 -c <server-ip>
```

Before the session, test that the Pi can ping the server:

```
ping <server-ip>
```

If students use Wi-Fi, results may vary a lot. That is okay, but they should mention it in the report.

8. Simple explanations you can give to beginners

CPU

The CPU is the part that does calculations. In this lab, we make it do many calculations and measure how long it takes.

RAM

RAM is temporary memory. When Python creates a big array, it uses RAM. When the program ends, that memory is released.

Storage

Storage is where files are saved permanently. The `dd` command writes and reads a test file to measure storage speed.

Network latency

Latency is delay. If ping is high, messages take longer to travel.

Network bandwidth

Bandwidth is capacity. Higher bandwidth means more data can be transferred per second.

Temperature

When the Pi works hard, it gets hotter. If it gets too hot, it may slow down to protect itself.

Throttling

Throttling means the Pi reduces performance because of heat or power problems.

Benchmark

A benchmark is a test used to measure performance.

9. Suggested lab flow for you

Start of lab

Tell students:

6. Create `results/`
7. Run each task
8. Save outputs
9. Write short explanation in `REPORT.md`
10. Commit and push to GitHub

Commands:

```
mkdir -p results
```

Recommended file creation:

```
nano cpu_test.py
nano memory_test.py
```

Run CPU test:

```
python3 cpu_test.py > results/cpu.txt
vcgencmd measure_temp >> results/cpu.txt
```

Run memory test:

```
python3 memory_test.py > results/memory.txt
free -h >> results/memory.txt
```

Run I/O test:

```
dd if=/dev/zero of=testfile.bin bs=100M count=1 oflag=direct 2>&1 | tee results/io.txt
dd if=testfile.bin of=/dev/null bs=100M count=1 iflag=direct 2>&1 | tee -a results/io.txt
rm testfile.bin
```

Run network test:

```
iperf3 -c <server-ip> > results/iperf.txt
ping <server-ip> -c 30 > results/ping.txt
```

Run thermal test:

```
stress --cpu 4 --timeout 30
```

In another terminal:

```
watch -n 2 vcgencmd measure_temp
```

For saving manual thermal results:

```
vcgencmd measure_temp >> results/thermal.txt
```

10. What `REPORT.md` should include

Students should not just dump command outputs. They should explain what happened.

A simple structure:

```
# Week 8 Report - Raspberry Pi Performance

## CPU Test
Execution time:
Temperature:
Observation:

## Memory Test
Memory allocation time:
```

```
Free memory before/after:
Observation:

## Storage I/O Test
Write speed:
Read speed:
Observation:

## Network Test
Bandwidth:
Ping min/avg/max:
Packet loss:
Observation:

## Thermal Test
Starting temperature:
Highest temperature:
Observation:

## Conclusion
What did you learn about Raspberry Pi performance?
```

Good observations:

- "The CPU temperature increased during stress."
- "The network bandwidth was lower on Wi-Fi than expected."
- "The SD card write speed was slower than read speed."
- "The Pi did not throttle during the 30-second test."
- "Results may vary because of cooling and background tasks."

Weak observations:

- "It worked."
- "The result was good."
- "Nothing happened."

Encourage them to explain numbers.

11. Short checklist for you before the lab

Before students arrive:

- Check at least one Raspberry Pi boots properly
- Check Python 3 works
- Check ``vcgencmd measure_temp`` works
- Install missing tools: ``htop``, ``iperf3``, ``stress``, ``python3-numpy``
- Choose one machine as the ``iperf3`` server
- Find the server IP address
- Test ``ping`` from Pi to server
- Test ``iperf3 -s`` and ``iperf3 -c <server-ip>``
- Make sure students know to create ``results/``
- Remind students to write ``REPORT.md``
- Remind students to commit and push their work

12. Missing/incomplete parts in the repo

The Week 8 repo is useful, but some parts are incomplete or could confuse students.

1. No installation instructions

The repo lists required tools but does not show how to install them.

Add or explain:

```
sudo apt update
sudo apt install htop iperf3 stress python3-numpy
```

2. `dd` redirection example may not capture output properly

The README shows:

```
dd ... > results/io.txt
```

But dd often outputs to stderr, so students may get an empty file.

Better:

```
dd ... 2>&1 | tee results/io.txt
```

3. Network server machine is not clearly defined

The repo says:

```
iperf3 -s
```

But students may not know what "server machine" means.

You should explain:

One device runs `iperf3 -s`. The Raspberry Pi connects to that device using `iperf3 -c <server-ip>`.

4. Thermal saving is vague

The repo says optional CSV, but does not give a clear command to automatically save temperatures.

You can give students a simple manual option:

```
vcgencmd measure_temp >> results/thermal.txt
```

Or a simple loop:

```
for i in {1..15}; do vcgencmd measure_temp >> results/thermal.txt; sleep 2; done
```

5. No clear report template

Students may not know what to write in `REPORT.md`.

Give them the structure from section 10.

6. No warning about cleanup

The I/O task creates:

```
testfile.bin
```

Students should delete it after the test:

```
rm testfile.bin
```

7. CPU test may finish too quickly on some devices

If the CPU test runs too quickly, students can increase:

```
cpu_stress(20_000_000)
```

But tell them not to go too high if the Pi is slow.

Your main role during Week 8

Your job is mostly to help students:

- Run commands correctly
- Save outputs correctly
- Understand what each metric means
- Debug missing tools
- Understand why results differ between groups
- Write a proper short report, not just paste raw outputs

The biggest confusion points will probably be:

11. Saving terminal output into files
12. Understanding `iperf3` server/client setup
13. Understanding bandwidth vs latency
14. Missing packages like `numpy`, `stress`, or `iperf3`
15. Interpreting numbers instead of just copying them